
BarkSwap ve(3,3) — Security Review

Final Report — Remediation Verified (commit 879e444)

pindarev



July 6, 2026

Contents

Disclaimer	3
About pindarev	3
Risk Classification	3
Protocol Summary	4
Audit Details	4
Scope	5
Roles	5
Executive Summary	6
Issues found	7
Remediation Status	9
Findings	9
Critical	9
[C-01] Anyone can steal every staker's farming rewards by force-updating their position	9
High	12
[H-01] A staked NFT can be locked in the gauge forever after a routine kill → clear . . .	12
Medium	14
[M-01] Rotating the external bribe mid-epoch strands the briber's funds	14
Low	15
[L-01] notifyRewardAmount re-spreads accumulated reserve over a single fresh week	15
[L-02] Fee sweep can divert reward-token balance to the bribe when reward == pool token	16
[L-03] Bribe address setters lack the code-shape guard every other setter enforces . .	16
[L-04] Bribe.notifyRewardAmount books the requested amount, not the amount received (FoT)	17
[L-05] Internal-bribe mid-epoch rotation can strand already-claimed fees (internal mirror of M-01)	18
Informational	19
[I-01] Anyone can register a gauge for a fake pool (known / accepted: R-6)	19
[I-02] Persistent vote weight never decays	19

[I-03] earned () reverts for every integrator if Algebra makes getRewardInfo non-view	20
[I-04] vote () silently burns the caller's whole epoch on all-invalid input	21
[I-05] getReward reverts for a position poked after a kill → clear	21
Appendix — reviewed and not elevated	22

Disclaimer

I make every effort to surface as many vulnerabilities in the code as possible within the given time period, but hold no responsibility for the findings provided in this document. This security review is not an endorsement of the underlying business or product. The work was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts in scope.

This report represents a snapshot of the codebase at the time of review. Subsequent changes to the code — including those made in response to this report — fall outside the scope of this engagement unless explicitly contracted for re-review.

About pindarev

pindarev is an independent blockchain security researcher with experience across audit contests and private engagements.

Channel	Link
Portfolio	github.com/PavelPindarev/audits
X / Twitter	@ppindarew
Telegram	@pindarev

Risk Classification

Severity is assessed using a standard impact × likelihood matrix:

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

- **Critical** — direct loss of user funds or permanent freezing of the protocol's primary functionality. Immediate remediation required.

- **High** — conditional loss of funds or permanent locking of user principal, reachable through a normal protocol lifecycle plus a permissionless trigger. Fix before launch.
- **Medium** — accounting drift, fairness violations, or stranded value gated by realistic preconditions (e.g. a semi-trusted role action). Fix before launch.
- **Low** — defensive hardening, narrow attack windows, or issues bounded by stated trust assumptions.
- **Informational** — design observations, future-proofing concerns, and best-practice recommendations.

Protocol Summary

BarkSwap is a ve(3,3) governance-and-emissions layer (forked from THENA's voter / minter / bribes) deployed over an Algebra Integral concentrated-liquidity DEX on DogeOS. Its defining departure from a standard ve(3,3) is the gauge: instead of a Synthetix-style LP-token gauge, BarkSwap uses a **custodial NFT gauge** (GaugeV2_NFT) that holds Algebra concentrated-liquidity position NFTs and delegates per-position reward accrual to Algebra's EternalFarming via the FarmingCenter.

The three custom contracts in scope are:

- **VoterV4** — persistent (Aerodrome-model) voter; gauge registry; emission distribution. Vote weights persist across epochs until the holder resets or re-votes.
- **GaugeV2_NFT** — NFT-custody gauge; delegates accrual to Algebra EternalFarming; exposes the IGauge surface to the voter.
- **GaugeFactoryV2_NFT** — non-upgradeable, immutably-wired factory for NFT gauges.

Emissions flow `Minter` → `VoterV4` → `GaugeV2_NFT` → `Algebra EternalFarming` (per-NFT, range-aware accrual is advanced by the pool's `afterSwap` hook). Bribes are per-epoch and are refreshed by the permissionless `poke` / `pokeMany`. The remaining ve(3,3) contracts (`VotingEscrow`, `Bribes`, `PermissionsRegistry`) are byte-identical to THENA upstream and were treated as out of the custom-code focus.

Audit Details

Field	Value
Auditor	pindarev
Engagement type	Private security review (responsible disclosure)

Field	Value
Review period	June 2026
Language	Solidity 0.8.13
Build framework	Hardhat
Test status	All proofs-of-concept executed against the in-scope code; pass
Remediation	Reviewed at commit 879e444; 7 resolved, 6 acknowledged

Scope

The following custom contracts were the focus of this review:

Contract	Path
GaugeV2_NFT	ve3/contracts/GaugeV2_NFT.sol
VoterV4	ve3/contracts/VoterV4.sol
GaugeFactoryV2_NFT	ve3/contracts/GaugeFactoryV2_NFT.sol

Out of scope:

- THENA mirror contracts (VotingEscrow, Bribes, PermissionsRegistry) — byte-identical to upstream after normalization and relied upon for prior audit coverage.
- The Algebra Integral core / plugin / farming / periphery — deployed upstream v1.2.1-integral and treated as a trusted dependency, **except** where its documented, in-the-deployed-bytecode behaviour is the enabler of an in-scope finding (notably the permissionless `NonfungiblePositionManager.increaseLiquidity` farming hook in C-01 / H-01).
- Off-chain components (keepers, indexers, subgraph, frontends).

Roles

Role	Powers & trust assumption
GOVERNANCE	whitelist/blacklist, createGauge, killGauge, reviveGauge, clearKilledGauge, recoverStrayNFT, initializeGaugeIncentive. Fully trusted (governance multisig).
VOTER_ADMIN	AddFactory/removeFactory/replaceFactory, setMinter, setBribeFactory, bribe rotations. Semi-trusted (governance multisig).
Gauge DISTRIBUTION slot	The VoterV4 proxy. setPool, initializeIncentive, notifyRewardAmount, teardownIncentive, recoverNFT, emergency mode. Contract-level gate.
minter slot	The MinterUpgradeable proxy. notifyRewardAmount to the voter.
Permissionless	deposit / withdraw / getReward, poke / pokeMany, createGauge (whitelisted pairs), distribute*. No trust assumption — anyone may call.

Executive Summary

This review identified **1 Critical**, **1 High**, **1 Medium**, **5 Low**, and **5 Informational** findings across the three custom contracts. Every finding was subsequently remediated or formally acknowledged by the team; see the **Remediation Status** section below.

The most consequential finding is structural and fully permissionless. The custodial NFT gauge pools every staker's rewards into a **single** Algebra Eternal Farming bucket keyed by the gauge address, and drains that bucket in full via `claimReward(..., 0)`. Because Algebra's `NonfungiblePositionManager.increaseLiquidity` is unauthenticated and triggers the farming hook, the headline findings follow directly from this one design decision:

- **C-01 (Critical)** — anyone can call `increaseLiquidity` on a victim's gauge-custodied NFT to force the victim's accrued reward into the shared bucket, then call `getReward` and sweep it. An attacker holding **no staked position** can steal the earned emissions of every staker in the gauge, repeatably, for the cost of gas and dust. Proven end-to-end on a real in-process Algebra stack: the attacker takes the victim's full earned reward; the victim is left with dust.
- **H-01 (High)** — after a routine `killGauge` → `clearKilledGauge` lifecycle, the same permissionless `increaseLiquidity` clears the FarmingCenter's `deposits[tokenId]`, after which the gauge's `withdraw` reverts permanently and `recoverNFT` refuses. A user's de-

posited position NFT — real principal — is locked in the gauge forever.

- **M-01 (Medium)** — a mid-epoch external-bribe rotation followed by a permissionless poke migrates the voter's weight to the new bribe instance, stranding the briber's funds in the old (now weightless) instance and earning the voters nothing for that epoch.

All Critical / High / Medium findings were demonstrated with executable Hardhat proof-of-concept tests on a real in-process Algebra stack; that exploit code is omitted from this status-of-record report and was delivered to the team separately. The remaining findings address defensive hardening (missing code-shape guards, reward-token-aware fee sweeps, fee-on-transfer accounting) and design / future-proofing observations (non-decaying persistent vote weight, the earned () view-shim's dependence on an upstream Algebra guarantee, a silent epoch lockout, and the team-accepted fake-pool registration).

Issues found

Severity	Count
Critical	1
High	1
Medium	1
Low	5
Info	5
Total	13

ID	Title	Severity	Status
C-01	Anyone can steal every staker's farming rewards by force-updating their position	Critical	Resolved
H-01	A staked NFT can be locked in the gauge forever after a routine kill → clear	High	Resolved
M-01	Rotating the external bribe mid-epoch strands the briber's funds	Medium	Resolved
L-01	not i f y R e w a r d A m o u n t re-spreads accumulated reserve over a single fresh week	Low	Acknowledged

ID	Title	Severity	Status
L-02	Fee sweep can divert reward-token balance to the bribe when reward == pool token	Low	Acknowledged
L-03	Bribe address setters lack the code-shape guard every other setter enforces	Low	Resolved
L-04	Bribe.notifyRewardAmount books the requested amount, not the amount received (FoT)	Low	Resolved
L-05	Internal-bribe mid-epoch rotation can strand already-claimed fees (internal mirror of M-01)	Low	Acknowledged
I-01	Anyone can register a gauge for a fake pool (known / accepted: R-6)	Info	Acknowledged
I-02	Persistent vote weight never decays	Info	Resolved
I-03	earned() reverts for every integrator if Algebra makes getRewardInfo non-view	Info	Acknowledged
I-04	vote() silently burns the caller's whole epoch on all-invalid input	Info	Resolved
I-05	getReward reverts for a position poked after a kill → clear	Info	Acknowledged

Remediation Status

The three custom contracts were remediated by the team in commit 879e444 (*fix(ve3): pindarev audit remediation*). Each fix was independently re-reviewed against the updated source, and for the Critical / High / Medium items re-verified with the remediation's own executable proof-of-concept suites on a real in-process Algebra stack — all pass, with no regressions in the surrounding logic.

Of the eleven findings in the original review, seven are **resolved** and four are **acknowledged** with rationale (documented, non-fund-loss trade-offs). The re-review surfaced two additional minor items — **L-05** and **I-05** — both low-impact and recoverable; they are recorded here for completeness and acknowledged.

Disposition	Findings
Resolved (879e444)	C-01, H-01, M-01, L-03, L-04, I-02, I-04
Acknowledged	L-01, L-02, I-01, I-03, L-05, I-05

This document is the status-of-record deliverable: exploit / proof-of-concept code has been omitted so the report is safe to distribute publicly. The complete executable PoCs were delivered to the team separately.

Findings

Critical

[C-01] Anyone can steal every staker's farming rewards by force-updating their position

Status: Resolved — commit 879e444. The gauge now custodies each depositor's NFTs under a dedicated per-user `PositionCustodian`, giving every depositor a private `EF.rewards[custodian]` bucket instead of one shared pot; a permissionless poke now materialises a victim's reward into the victim's own bucket, which only they can drain. Independently re-reviewed and confirmed on a real Algebra stack — the attacker nets zero and depositors keep their full reward.

Description

When a user deposits an Algebra position NFT into the gauge, the gauge becomes the owner of that NFT. Algebra's eternal farming credits rewards *per owner*, so because the gauge owns every deposited NFT, the accrued rewards of **all** stakers land in one shared bucket keyed by the gauge address: `EF.rewards[gauge][rewardToken]`.

The gauge assumes this shared bucket is always emptied in the same transaction it is filled. Its own functions (`getReward`, `_withdraw`) collect a user's rewards into the bucket and immediately drain it — so between calls the bucket is meant to be 0. The project even ships a test, `GaugeV2_NFT_INV_POT_proof.test.ts`, asserting this “pot is always 0 between calls” invariant. **The invariant is never enforced on-chain.**

The drain uses `EF.claimReward(rewardToken, recipient, 0)`. Passing 0 means “send the entire bucket”, not “send my share” — only safe while the bucket holds nothing but the caller's own freshly-collected rewards.

```
// GaugeV2_NFT.getReward(address account)
for (uint256 i = 0; i < ids.length; i++) {
    FC.collectRewards(key, ids[i]);           // collects only `account`'s NFTs
}
uint256 pending = EF.rewards(address(this), IERC20Minimal(address(rewardToken)));
if (pending > 0) {
    EF.claimReward(IERC20Minimal(address(rewardToken)), account, 0); // ← drains the
    ↪ WHOLE bucket
}
```

The break is that an attacker can fill the bucket with *someone else's* rewards without draining it, using a permissionless `Algebra` call. `NonfungiblePositionManager.increaseLiquidity` has **no owner check** (`NonfungiblePositionManager.sol:252`) — anyone can add dust liquidity to anyone's position. Doing so on a victim's staked NFT triggers the farming hook (`FarmingCenter._updatePosition`), which calls `EF.exitFarming(key, victimNFT, owner = gauge)`, materialising the victim's full accrued reward into `rewards[gauge][rewardToken]`, then re-enters farming. No drain follows, so the victim's reward simply sits in the shared bucket. The attacker then calls `getReward(attacker)` and `claimReward(..., 0)` ships the whole bucket to them.

Impact

Anyone, with no role and no staked position, can steal the pending farming rewards of every other staker in the gauge. The cost is gas plus a dust amount of liquidity — which is added to the victim's own position, so the attacker risks essentially nothing. It is repeatable every block: a bot can force-update + drain continuously, so honest LPs never receive their emissions and a single attacker can capture 100% of a gauge's BARK emissions indefinitely. This is direct theft of funds, not merely a denial of service, and it defeats the core purpose of the gauge.

Recommended mitigation

Stop draining the whole bucket. Claim exactly the amount the caller's own NFTs just produced, summing the `per-tokenId` return values, and never pass 0:

```

uint256 owed;
for (uint256 i = 0; i < ids.length; i++) {
    (uint256 collected, ) = FC.collectRewards(key, ids[i]); // this account's NFTs
    ↪ only
    owed += collected;
}
if (owed > 0) {
    EF.claimReward(IERC20Minimal(address(rewardToken)), account, owed); // never 0
}

```

Apply the same change to `_withdraw` (capture the `exitFarming` return for the single `tokenId`) and remove the shared-pot add in `earned()` (`GaugeV2_NFT.sol:210`). This is the **immediate** mitigation: it stops the theft, because an attacker who claims only the sum of their own NFTs' freshly-collected amounts can no longer take a victim's materialised reward.

Does this fully fix the problem, and does it open a new one? No — it is a stop-gap, not a complete fix, and the team must understand the limit. The minimal patch converts the bug from **theft** into **stranding**: when a permissionless `increaseLiquidity` poke materialises a victim's accrual into the shared bucket, `collectRewards` afterwards returns only the accrual *since* that poke, so the victim's poked-out amount sits in the shared EF bucket and is claimable by no one under per-NFT accounting.

That residue cannot be reclaimed on-chain, and the reason is structural: **as long as every staked NFT shares a single EF reward owner (the gauge) and `increaseLiquidity` can permissionlessly reset any NFT's EF accumulator, the poke erases the on-chain record that the materialised amount belonged to that NFT — no gauge-side bookkeeping can re-attribute it, because the gauge is never notified of the external materialisation.** A complete fix is therefore a design change, not a one-line patch, with two viable directions:

1. **Isolate each position's reward owner (recommended).** Custody each NFT under a distinct, depositor-bound owner (a per-NFT/per-user minimal custodian) so EF credits rewards[thatOwner][token] — a private bucket. A poke then materialises the victim's reward into *their own* bucket, which only they can drain; cross-user theft and stranding both disappear because there is no shared pot. The cost is one custodian per position (gas + a registry on the gauge); the pool-level incentive funded by `notifyRewardAmount` is unchanged.
2. **Make the gauge the source of truth.** Move per-user reward accounting onto the gauge and treat EF purely as a funding reservoir. This is harder than it sounds: accrual is range-aware (the reason EF was used), so the gauge would have to snapshot every NFT's `getRewardInfo` on every interaction — and it still cannot capture a materialisation that happens *between* interactions. Direction (1) is cleaner.

Whichever direction is taken, **do not add the `require(EF.rewards(this, token) == 0)` runtime guard that the project's own test suggests.** Because a permissionless poke makes a non-zero pot a reachable, attacker-controlled state, that guard would let anyone brick `getReward / _withdraw` on demand — a denial-of-service strictly worse than the stranding. The INV-POT assertion belongs only in tests, never in the contract. The recommended path: ship the claim-own-amount patch now to stop active theft, then scope and re-review direction (1) to close the residue.

High

[H-01] A staked NFT can be locked in the gauge forever after a routine kill → clear

Status: Resolved — commit 879e444. Path A is closed by a new `emergencyWithdraw` whose principal return never depends on `FC.exitFarming` (best-effort reward settle, unconditional NFT return to the recorded owner, state-gated so it cannot bypass a healthy `withdraw`). Path B is closed by making `initializeIncentive` one-shot (`require(virtualPool == 0)`), so a retired gauge's `incentiveKey` can never be overwritten. Re-reviewed and confirmed.

Description

Governance retires a gauge with `killGauge` then `clearKilledGauge`. The clear step calls the gauge's `teardownIncentive`, which deactivates the Algebra farming program. The gauge deliberately preserves `incentiveKey` so that users whose NFTs are still custodied can still call `withdraw` and `exit` — this is the documented INV-3 guarantee.

That guarantee is fragile. Deactivating the incentive disconnects the virtual pool inside the `FarmingCenter` (`virtualPoolAddresses[pool] = 0`). Anyone then calls the permissionless `NPM.increaseLiquidity(stakedNFT, dust)`. The farming hook sees the disconnected pool and fully exits the position, clearing `deposits[tokenId] = 0` (`FarmingCenter.sol:70`). After that the user's `withdraw` is permanently bricked:

```
// GaugeV2_NFT._withdraw → FarmingCenter.exitFarming
require(deposits[tokenId] == IncentiveId.compute(key), 'Invalid incentiveId'); //
↪ deposits == 0 → revert
```

`recoverNFT` cannot help (it refuses any NFT that is an accounted-for stake, `tokenOwner != 0`), and there is no emergency exit. The NFT is stuck.

There are two ways to reach this state, both rooted in tearing the incentive down while NFTs are custodied:

- **Path A — permissionless (anyone):** the `increaseLiquidity` poke above clears `deposits[tokenId]`.
- **Path B — governance misstep:** `clearKilledGauge` leaves `isGauge[_gauge] == true` (VoterV4.sol:715). Because `initializeGaugeIncentive` only checks `isGauge` (VoterV4.sol:746), governance can re-initialise the old, cleared gauge instead of the fresh one; `initializeIncentive` only guards `!incentiveInitialized` (tear-down reset it), so it overwrites `incentiveKey` with a new nonce — and every still-custodied NFT then fails `_withdraw` against the new key.

Impact

A user's deposited position NFT — real principal, not just rewards — can be permanently locked in the gauge. The Path A trigger is a normal governance lifecycle (kill → clear a pool to re-gauge it) followed by a permissionless dust call, so anyone can grief every still-staked NFT, or it can happen accidentally the moment a user touches their position. It breaks the exact INV-3 guarantee the code claims to provide, with no recovery path.

Recommended mitigation

Add a true emergency exit that returns a custodied NFT to its owner without routing through `FC.exitFarming`, so it works even after `deposits[tokenId]` is cleared; and clear `isGauge[_gauge]` in `clearKilledGauge` (or guard `initializeIncentive` with `require(stakedTokenIds.length == 0)`) so a cleared gauge cannot be re-initialised into an `incentiveKey` overwrite.

Does this fix the problem without opening a new one? The emergency exit must be carefully scoped, because a naive “owner can pull any NFT out” function would itself be a new vector — it could be used to bypass the normal `withdraw` flow (skipping fee collection / reward settlement) during normal operation. Gate it so it is callable **only after the incentive is torn down / deactivated**, restrict it to the position's recorded `tokenOwner` (or governance acting on their behalf), and have it best-effort call `FC.exitFarming` first and fall back to a raw `safeTransferFrom` only on revert — while still cleaning up the gauge's own bookkeeping (`tokenOwner`, `userTokenIds`, `stakedTokenIds`) so accounting stays consistent. The simpler alternative, `require(stakedTokenIds.length == 0)` before teardown, fully prevents the bug but introduces a liveness trade-off: governance cannot re-gauge a pool until every staker has withdrawn, so an apathetic or malicious staker can block re-gauging indefinitely — acceptable only if paired with a way to force-return those NFTs. Finally, note that clearing `isGauge` in `clearKilledGauge` is safe but has one side effect: `recoverStrayNFT` and the bribe setters gate on `isGauge`, so they will no longer operate on the cleared gauge — intended, since the gauge is dead, but worth confirming no operational flow relies on them post-clear.

Medium

[M-01] Rotating the external bribe mid-epoch strands the briber's funds

Status: Resolved — commit 879e444. `_pokeBribes` now keeps each vote's external weight on the instance it was bound to at vote time instead of migrating it to the current pointer, so a mid-epoch rotation no longer strands a funded briber. Re-reviewed and confirmed; the residual (a pot funded on the rotated-in instance before any re-vote) is not attacker-triggerable and is recoverable via `recoverERC20AndUpdateData`.

Description

`setExternalBribeFor` lets a `VoterAdmin` point a gauge at a new external-bribe contract (`EB_new`). The permissionless poke then, inside `_pokeBribes`, withdraws the voter's weight from the recorded old instance (`EB_old`) and re-deposits it into the current instance (`EB_new`) for the same epoch slot:

```
// VoterV4._pokeBribes (external leg)
address _recEb = voteExternalBribe[_tokenId][_pool];
if (_recEb != address(0)) IBribe(_recEb).withdraw(_amt, _tokenId); // remove from
    ↪ EB_old
address _curEb = external_bribes[_gauge];
IBribe(_curEb).deposit(_amt, _tokenId); // add to EB_new
```

If a briber has already funded `EB_old` for the upcoming epoch, the rotation + poke leaves `EB_old` holding the briber's reward tokens with zero backing weight (nobody can earn them), while `EB_new` holds the weight but no rewards. The briber's funds are stranded and the voters who backed that gauge earn nothing for that epoch.

Impact

A briber's tokens for the funded epoch become permanently stranded in the old bribe contract — recoverable only via a governance escape hatch — and the voters who backed that gauge earn nothing for the epoch. The trigger requires a `VoterAdmin` rotation (semi-trusted), amplified by the permissionless poke. The code itself flags this desync as a known separate finding (`VoterV4.sol:1121-1122`).

Recommended mitigation

The most robust fix is to keep the current epoch's accounting bound to the instance the vote was deposited into: have `_pokeBribes` re-deposit into the **recorded** `voteExternalBribe` instance for the current epoch (the A6 binding already stores it), and only let a *fresh* `vote()` move weight to the new instance.

Does this fix the problem without opening a new one? It removes the stranding, but it must be weighed against the reason rotations happen. If a VoterAdmin rotates specifically to escape a broken or malicious EB_old, then continuing to deposit/withdraw against EB_old in poke re-introduces a denial-of-service: a reverting EB_old would brick poke for every voter still bound to it. So the safer overall design is to (a) add a voter-callable migrate(tokenId) on the Bribe contract that atomically moves both _balances[owner][slot] and _totalSupply[slot] for the live slot from old to new (the Voter cannot write another contract's storage, so the migration must live on Bribe), guarding against double-migration; or (b) constrain rotations operationally to epoch boundaries before any funding lands, and document it. Avoid the naive “forbid rotation while the slot is funded” check — it cannot be enforced cheaply on-chain because the Bribe cannot enumerate all reward tokens/epochs in setExternalBribeFor.

Low

[L-01] notifyRewardAmount re-spreads accumulated reserve over a single fresh week

Status: Acknowledged — retained by design. The team documents the full-reserve-over-DURATION behaviour as intentional (it avoids under-distribution in low-liquidity weeks); Distribution-gated, no fund loss.

Description

GaugeV2_NFT.notifyRewardAmount recomputes the emission rate from the *entire* remaining reserve over a fresh 7-day window:

```
(uint128 reserveAfterAdd,) = virtualPool.rewardReserves();
uint256 newRate = uint256(reserveAfterAdd) / DURATION; // full reserve, incl.
↳ undistributed leftover
EF.setRates(incentiveKey, uint128(newRate), 0);
```

If distribution is skipped for several weeks (reachable through the kill / skip-list operational paths), the piled-up reserve is front-loaded into a single week as an emission spike, rather than being spread at its prior rate.

Impact

A temporary emission-schedule distortion (a one-week spike after a multi-week gap). It is only-Distribution-gated, so it is not directly attacker-triggerable, and there is no fund loss — only a deviation from the intended smooth schedule.

Recommended mitigation

Schedule only the newly-added reward over DURATION, or carry the prior leftover forward at its existing rate rather than re-dividing the whole reserve. *Mitigation safety:* carrying leftover forward at the existing rate is purely arithmetic and introduces no new state or external interaction; ensure rounding of the carried remainder does not silently truncate emissions to zero for very small notifies.

[L-02] Fee sweep can divert reward-token balance to the bribe when reward == pool token

Status: Acknowledged — dust-bounded. `notifyRewardAmount` is atomic (transfer-in → addRewards-out) so the gauge holds no meaningful reward-token balance between calls; the suggested carve-out was judged to add more risk (stranding real fees) than it removes.

Description

`claimFees / flushFees / sweepFees` forward the gauge's full `token0 / token1` balance to the internal bribe (`GaugeV2_NFT.sol:490-537`). If `rewardToken` equals one of the pool tokens, any reward-token balance held by the gauge is swept along with the fees.

Impact

Bounded to dust: `notifyRewardAmount` is atomic (transfer-in → addRewards-out), so the gauge never holds a meaningful reward-token balance between calls. The realistic exposure is the 1-wei seed and rounding dust.

Recommended mitigation

When `rewardToken` coincides with a pool token, carve it out of the fee-forwarding sweep (forward `balanceOf(token) - reservedReward`). *Mitigation safety:* the carve-out only triggers on the `rewardToken == poolToken` deployment and does not affect the normal case, so it cannot strand legitimate fees; confirm the reserved amount tracks only protocol-owned reward dust, not user fees.

[L-03] Bribe address setters lack the code-shape guard every other setter enforces

Status: Resolved — commit 879e444. `setInternalBribeFor / setExternalBribeFor / setNewBribes` now reject non-contract pointers (`if (_x.code.length == 0) revert NotContract();`), matching every other setter.

Description

Every other address setter in VoterV4 enforces `code.length > 0`, but `setInternalBribeFor`, `setExternalBribeFor` and `setNewBribes` (VoterV4.sol:1100-1124) do not. The value set here is later called by the permissionless `poke` / `pokeMany`.

Impact

A VoterAdmin typo (EOA or undeployed address) sets a bribe pointer that `poke` later calls, reverting with a CALL to a non-contract and bricking `poke` for that gauge until corrected. The worst proven outcome is a recoverable DoS, not theft.

Recommended mitigation

Add `if (_bribe.code.length == 0) revert NotContract();` to all three setters, matching `setMinter` / `setBribeFactory` / `setPermissionsRegistry`. *Mitigation safety*: it mirrors an existing, proven pattern and rejects only non-contract addresses; deployed bribe contracts (including proxies) carry bytecode, so no legitimate configuration is blocked.

[L-04] `Bribe.notifyRewardAmount` books the requested amount, not the amount received (FoT)

Status: Resolved — commit 879e444. `Bribe.notifyRewardAmount` now books the measured `balanceOf` delta across the transfer rather than the requested amount, so a fee-on-transfer reward token can no longer over-book `rewardsPerEpoch`.

Description

`Bribe.notifyRewardAmount` performs `safeTransferFrom` then books the full reward parameter into `rewardsPerEpoch` without measuring the balance actually received (Bribes.sol:330-343). For a fee-on-transfer / deflationary reward token the contract receives less than it books. (Bribes.sol is byte-identical to THENA; reaching this requires governance to whitelist the token as a reward token.)

Impact

`rewardPerToken` over-distributes and the last claimant's `safeTransfer` reverts, so the tail of every claim set becomes permanently unclaimable for that token. It requires a trusted action (whitelisting a FoT token), so the likelihood is low.

Recommended mitigation

Measure the balanceOf delta across the transfer and credit the delta rather than the requested reward. *Mitigation safety:* the delta-measurement pattern is standard; notifyRewardAmount is already nonReentrant, so the before/after read cannot be manipulated mid-call, and the change is conservative (it can only credit $\leq$ the requested amount).

[L-05] Internal-bribe mid-epoch rotation can strand already-claimed fees (internal mirror of M-01)

Status: Acknowledged — operational trade-off; funds recoverable via recoverERC20AndUpdateData. Semi-trusted trigger, bounded to one epoch's fees.

Description

_pokeBribes handles the internal (fee-sink) leg by migrating the vote's weight to the current internal_bribes [gauge] pointer on every poke — deliberately, so the live LP-fee stream routes to the current instance (this is why the M-01 remediation fixed only the *external* leg). The consequence is the symmetric case: if claimFees() has already funded the *old* internal-bribe instance for the current epoch slot and a VoterAdmin then calls setInternalBribeFor(gauge, EB_int_new), a subsequent permissionless poke withdraws the vote's weight from the old instance and re-deposits it into the new one — leaving the fees already booked on the old instance with zero backing weight.

Impact

A single epoch's LP fees that were claimed into the old internal-bribe instance before a mid-epoch rotation become unclaimable on that instance (no backing weight to divide by). The trigger is a semi-trusted VoterAdmin rotation amplified by a permissionless poke; the exposure is bounded to fees booked in the same epoch as the rotation, and the funds are recoverable by governance via Bribe.recoverERC20AndUpdateData. There is no permissionless theft — this is the internal-leg mirror of the accepted M-01 external residual, and predates the remediation (it is inherent to the A6 migrate-on-poke design, not introduced by the fix).

Recommended mitigation

Apply the same operational runbook the team documented for setExternalBribeFor to setInternalBribeFor — rotate at an epoch boundary, before fees accrue — or document the recoverERC20AndUpdateData recovery path on the setter. No contract change is required if the operational constraint is accepted.

Informational

[I-01] Anyone can register a gauge for a fake pool (known / accepted: R-6)

Status: Acknowledged — pre-agreed as R-6. Mitigated by the token whitelist + governance kill-Gauge; a stricter pool check would break Algebra custom-deployer pools. Documentation only.

Description

When creating an NFT gauge, the voter trusts the pool's self-reported `factory()` (`VoterV4.sol:819`) instead of asking the trusted factory whether the pool is real. A non-governance caller can deploy a fake "pool" returning two whitelisted token addresses and the configured factory address, and the voter will create a gauge for it. The team already documents this as **R-6 fake-pool spam** (`VoterV4_CreateGauge.audit.test.ts:53-60`) and has accepted it / will not fix — it is mitigated by the whitelist + governance oversight, and a stricter check would break Algebra's legitimate custom-deployer pools. Reported for traceability.

Impact

A non-governance attacker can spam junk gauges for fake pools, growing `pools[]` and bricking the convenience `distributeAll / distribute(range)` paths (fail-fast) until governance kills the junk gauge. No theft — a junk gauge cannot be incentive-initialised (governance-gated). Bounded by the §7.4 skip-list.

Recommended mitigation

Validate the pool against the Algebra pool deployer (`PoolAddress.computeAddress`) or the factory's own registry rather than the pool's self-reported `factory()`. *Mitigation safety:* `computeAddress` keyed on the pool's deployer validates custom-deployer pools correctly (unlike the strict `poolByPair` lookup the team rejected as their "M-9"), so it closes the spoof without re-introducing that regression. The team has accepted the residual risk; this is documentation only.

[I-02] Persistent vote weight never decays

Status: Resolved — commit 879e444. A new permissionless `resetExpired(tokenId)` retires the frozen weight of a fully-expired lock (`locked__end != 0 && <= block.timestamp`); live-lock decay is deliberately unchanged (re-basing on poke would re-open the keeper-griefing vector). Re-reviewed and confirmed.

Description

Emissions and bribes use the frozen cast-time weights[pool] / stored votes indefinitely; poke re-deposits the frozen weight each epoch even as the veNFT's voting balance decays (VoterV4.sol:486,550). A single peak-power vote therefore captures a permanent emission/bribe share without re-voting. This is a deliberate choice per §7.2 (the old poke re-based to decayed power and was abused), flagged to confirm it matches the intended Aerodrome persistent-voting semantics.

Impact

No fund loss or attacker path; a design property. The concern is fairness drift: long-standing votes keep their original weight while newer voters' real power decays, which may not match the protocol's intended emissions distribution.

Recommended mitigation

Confirm intent. If decaying influence is desired, re-evaluate weight against the current balance-ofNFT on poke. *Mitigation safety:* note that this re-opens exactly the griefing vector §7.2 closed (any keeper could erode a rival's allocation by poking it to its decayed value), so it should only be changed alongside a guard that restricts who can refresh a given tokenId — otherwise the “fix” is a regression.

[I-03] earned() reverts for every integrator if Algebra makes getRewardInfo non-view

Status: Acknowledged — latent and externally gated. Fires only if a future Algebra upgrade makes getRewardInfo state-writing; the deployed farming implementation is pinned and a compile-time guard (EarnedViewProbe.sol) fails the build if the view assumption breaks. No on-chain impact.

Description

earned() is declared view, but Algebra's getRewardInfo is declared non-view in its interface. The gauge works around this by casting EF to a local IEternalFarmingViewShim that re-declares getRewardInfo as view (GaugeV2_NFT.sol:25-30, 200-208), so earned() calls it via STATICCALL. This holds only because the currently deployed implementation is read-only; it is compile-guarded by EarnedViewProbe.sol, not protected at runtime.

Impact

If a future Algebra farming upgrade makes getRewardInfo write state, every STATICCALL from earned() reverts, breaking earned() for all off-chain consumers (frontends, keepers, integrators). On-chain fund flows are unaffected — the gauge never calls earned() internally. Latent and dependent on an out-of-scope Algebra change.

Recommended mitigation

Pin/own the farming implementation, or expose a non-view `earnedWrite()` alongside the view shim so integrators have a guaranteed-correct path if the upstream view guarantee changes. *Mitigation safety:* a non-view `earnedWrite()` must not be relied upon by any on-chain view consumer (it cannot be called from a `STATICCALL` context), so document it as off-chain/simulation-only; it introduces no new privileged surface.

[I-04] `vote()` silently burns the caller's whole epoch on all-invalid input

Status: Resolved — commit 879e444. `vote()` now reverts `NoLiveVote` when no live weight is recorded, rolling back the internal `_reset` so a fully-invalid submission neither consumes the epoch nor wipes the prior standing vote. Re-reviewed and confirmed.

Description

`vote()` bumps `lastVoted[_tokenId] = block.timestamp` unconditionally, even when `_vote` records zero live votes — empty arrays, all-dead/killed gauges, unregistered pools, or orphan pools where `clearKilledGauge` zeroed gauges `[_pool]` (`VoterV4.sol:534-539, 550-614`). The `onlyNewEpoch` gate then locks the caller out for the rest of the epoch. This mirrors `THENA VoterV3` and is documented as accepted (§7.3, “UI is expected to filter”).

Impact

A user who submits a vote that records nothing (stale UI, a pool killed between read and submit, or a griever who gets a pool killed mid-epoch) loses their entire epoch of voting / bribe participation with no on-chain feedback. No fund loss; a UX footgun.

Recommended mitigation

Revert when `_usedWeight == 0` (no live vote recorded) instead of bumping `lastVoted`, so a no-op vote does not consume the epoch. *Mitigation safety:* `reset()` is a separate path, so reverting an empty vote `()` does not interfere with legitimate abstention; the only behavioural change is that a fully-invalid vote now fails loudly instead of silently — strictly safer for the user.

[I-05] `getReward` reverts for a position poked after a kill → clear

Status: Acknowledged — graceful-degradation gap; no fund loss, recoverable via emergency-`Withdraw`.

Description

After `killGauge → clearKilledGauge` deactivates the incentive, a permissionless `NonfungiblePositionManager.increaseLiquidity` on a still-custodied NFT causes Algebra's farming hook to fully exit the position (it cannot re-enter a deactivated incentive), setting the farm's liquidity to zero. A subsequent `getReward(account)` calls `FarmingCenter.collectRewards` for that token, which routes to `AlgebraEternalFarming.collectRewards → _getFarm`, and `_getFarm` reverts `farmDoesNotExist()` when the farm's liquidity is zero. The convenience harvest therefore reverts for that user in this specific retired-gauge state. (Verified against the deployed Algebra bytecode.)

Impact

No fund loss and no impact on other stakers — `getReward` is per-account and iterates only the caller-account's own tokens. The affected user still recovers both the NFT and any already-materialised reward via `emergencyWithdraw`, which wraps `FC.exitFarming` and the custodian-bucket claim in `try/catch`. The issue is confined to the killed-and-cleared-and-poked edge and is a graceful-degradation gap in the convenience path, not a loss of principal.

Recommended mitigation

Wrap the per-token `FC.collectRewards(key, ids[i])` call inside `getReward` in a `try/catch` (mirroring `emergencyWithdraw`), so a single exited position cannot revert the whole harvest. Off-chain consumers should treat a poked, retired-gauge position as an `emergencyWithdraw` case.

Appendix — reviewed and not elevated

The following were investigated against the source and either reduce to a known/accepted item, are latent, or are self-DoS only — recorded for completeness:

- **\$7.7 stale-vote bribe orphan** — on the stale `_reset` path the per-NFT bribe deposit is not withdrawn; the depositor keeps their own one-epoch entitlement (per-pool `_totalSupply`, so no dilution of other voters). Benign, team-acknowledged.
- **_deposit cached-liquidity staleness** — `tokenLiquidity[tokenId]` is a one-time snapshot used only as a `> 0` gate; reward math uses live farm liquidity. Non-issue.
- **_vote divide-by-zero** — all-dead input reverts the caller's own `vote()`; self-DoS, retryable.
- **veNFT transfer vs owner-keyed bribe** — verified safe: `VotingEscrow._transferFrom` blocks transfer while voted (`VotingEscrow.sol:286`), so balances cannot orphan via transfer.

- **killGauge un-settled delta** — verified by-design: no cross-gauge mis-credit; the killed pool's already-notified share is forfeited (stranded in the Voter). Documented forfeit-on-kill.
- **THENA mirror files** — RewardsDistributor re-lock boundary revert + ve_for_at dead clamp, PermissionsRegistry.removeRole view-only reverse-index corruption, Bribe per-claim rounding dust. Inherited from THENA, out of the custom-code focus.